

Лекция 1. Дисковая подсистема Linux-сервера: LVM и программный RAID

Цель лекции

Понять, как Linux выстраивает путь от физического диска до каталога приложения

Объяснять назначение LVM и программного RAID в серверной инфраструктуре

Создавать, расширять и проверять логические тома и файловые системы

Оценивать состояние RAID, дисков, свободного места и inode

Безопасно настраивать автоматическое монтирование по UUID и проверять результат

Учебные вопросы

1. Архитектура хранения: диск, раздел, блочное устройство, файловая система и точка монтирования
2. LVM: PV, VG, LV; создание, форматирование и расширение томов и файловых систем
3. LVM snapshot, возможности и ограничения
4. RAID1, RAID5 и RAID10; состояния массива в Linux
5. SMART, /proc/mdstat, mdadm, df, du, inode
6. Монтирование по UUID и проверка /etc/fstab
7. Заполнение диска, нехватка inode, отказ диска и ошибка монтирования

1. Хранение данных в Linux строится слоями

Администратор работает не с одним объектом, а с цепочкой уровней хранения

Disk - физический или виртуальный накопитель, например /dev/sdb

Partition - раздел диска, например /dev/sdb1

Block device - блочное устройство, из которого ОС читает и в которое пишет блоками

Filesystem - структура данных, которая превращает блоки в каталоги и файлы

Mount point - каталог, через который файловая система подключается к дереву Linux

Диск и раздел не являются файловой системой

Диск предоставляет пространство, но сам по себе не хранит файлы в виде каталогов

Раздел задает границы области на диске и может иметь тип для Linux, RAID или LVM

Файловая система ext4 или XFS создается командой форматирования на выбранном устройстве

Каталог `/srv/apps` становится местом доступа к данным только после `mount`

Ошибка уровня хранения часто возникает из-за путаницы между устройством, ФС и точкой

монтирования

Точка монтирования определяет, куда попадут данные приложения

Если `/srv/apps` смонтирован, файлы записываются на отдельную файловую систему

Если `/srv/apps` не смонтирован, приложение может начать писать в обычный каталог корневого раздела

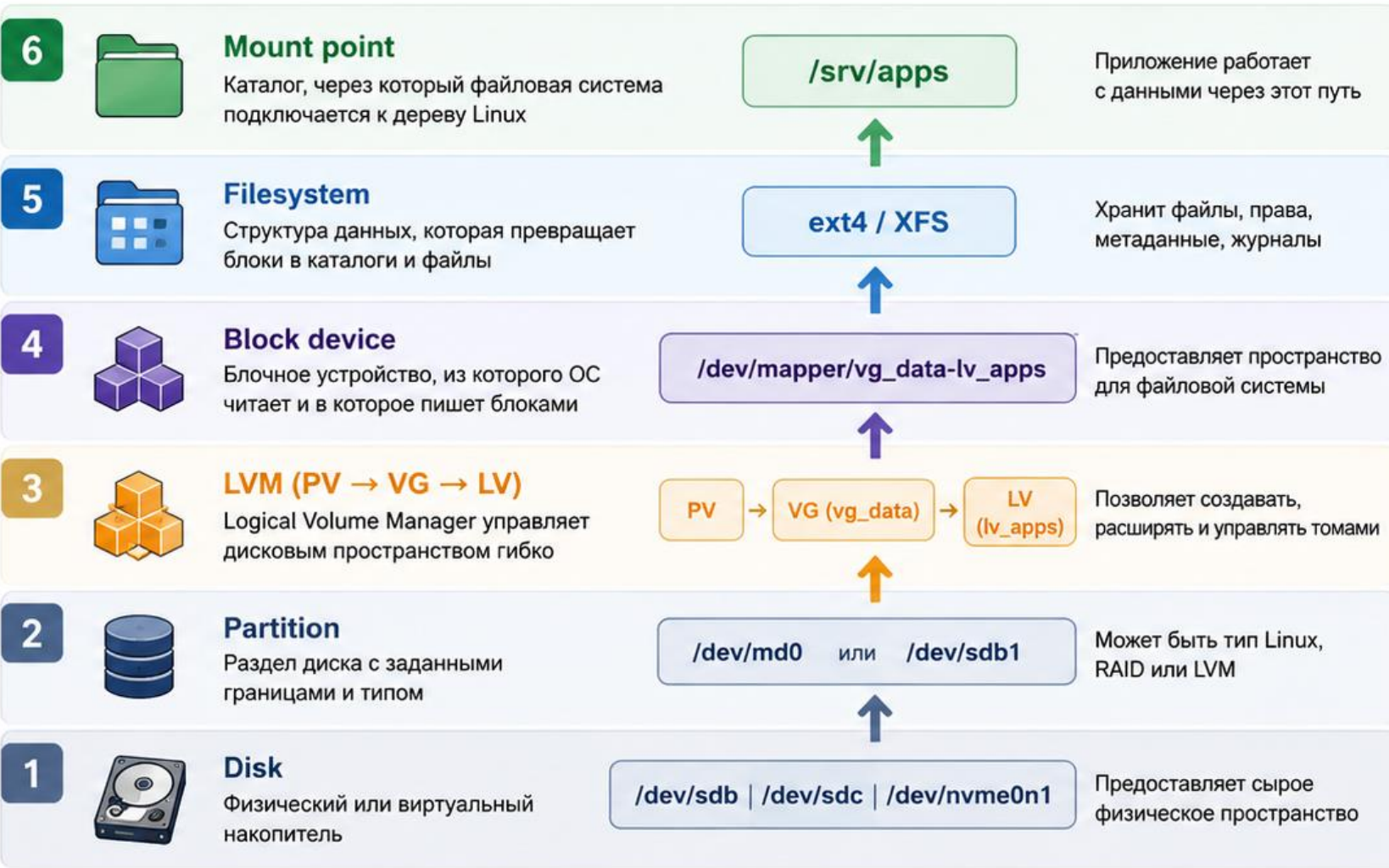
Команда `findmnt /srv/apps` показывает, какое устройство реально подключено к каталогу

Команда `df -hT /srv/apps` показывает тип ФС и свободное место именно для этой точки

После настройки хранения проверка `mount point` обязательна, иначе можно не заметить запись не туда

СЛОИ ХРАНЕНИЯ ДАННЫХ В LINUX

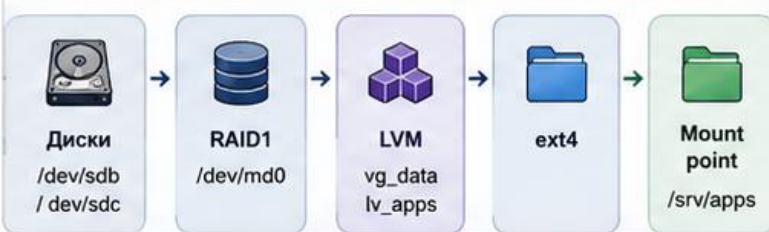
Путь от физического диска до каталога приложения



КАК ЧИТАТЬ СХЕМУ

- ↑ **Данные идут СВЕРХУ ВНИЗ при записи**
Приложение → файлы → блоки → устройство → диск
- ↓ **Данные идут СНИЗУ ВВЕРХ при чтении**
Диск → устройство → блоки → файлы → приложение
- 💡 **Каждый слой имеет свою задачу**
Путаница между устройством, ФС и точкой монтирования — частая причина ошибок

ПРИМЕР ЦЕПОЧКИ ДЛЯ /srv/apps



ПОЛЕЗНЫЕ КОМАНДЫ

- | | |
|--------------------------------|---|
| <code>lsblk -f</code> | Показать диски, разделы, LVM и точки монтирования |
| <code>blkid</code> | Показать UUID и тип файловой системы |
| <code>fdisk -l</code> | Показать таблицу разделов и размеры дисков |
| <code>findmnt /srv/apps</code> | Показать, что смонтировано в каталоге |
| <code>df -hT /srv/apps</code> | Показать тип ФС и свободное место |



ВАЖНО ПОМНИТЬ



Проверяйте точку монтирования после настройки хранения



Ошибки часто связаны с тем, что данные пишутся не туда, куда ожидается



RAID защищает от отказа дисков, LVM даёт гибкость, а файловая система хранит данные



Планируйте, проверяйте и регулярно контролируйте состояние системы хранения

В сервере RAID и LVM часто работают вместе

RAID отвечает за отказоустойчивость физических дисков

LVM отвечает за гибкое распределение пространства между томами

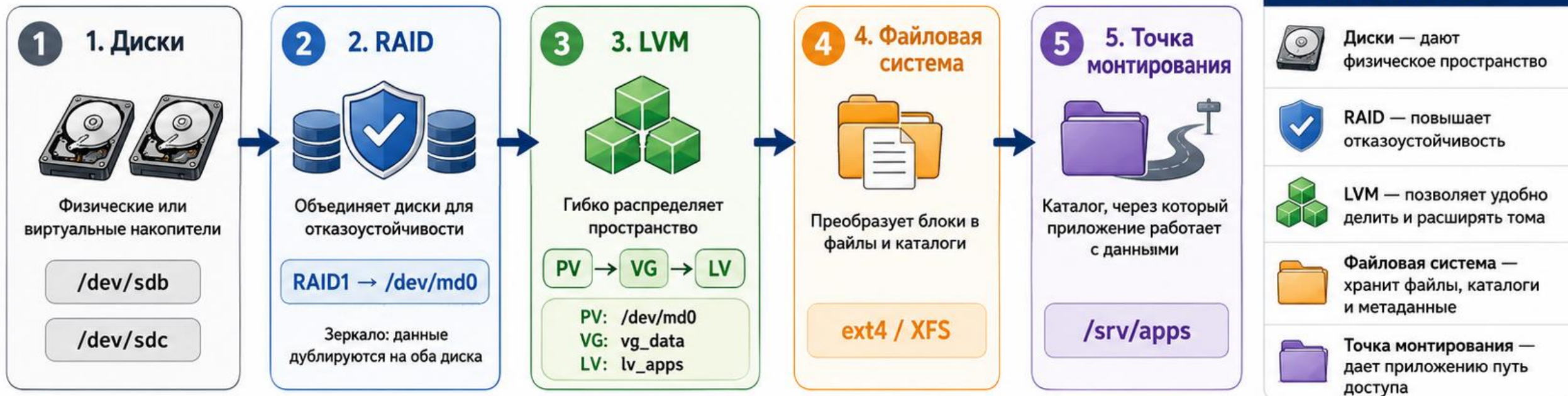
Файловая система отвечает за каталоги, файлы, права и метаданные

Mount point дает приложению стабильный путь, например /srv/apps

Типовая цепочка: диски -> RAID /dev/md0 -> PV -> VG -> LV -> ext4 -> /srv/apps

ДИСКИ, RAID, LVM, ФАЙЛОВАЯ СИСТЕМА И ТОЧКА МОНТИРОВАНИЯ

Как данные проходят путь от физических дисков до каталога приложения



ПРИМЕР ЦЕПОЧКИ



ПОЛЕЗНЫЕ КОМАНДЫ

lsblk +f



диски, разделы,
LVM, точки монтирования

cat /proc/mdstat



состояние RAID

pvs | vgs | lvs



состояние LVM

blkid



UUID и тип ФС

findmnt /srv/apps



что смонтировано
в каталог

df -hT /srv/apps



тип ФС и свободное место



RAID ≠ backup

RAID защищает от отказа диска, но не от удаления данных, вирусов и ошибок пользователей.



LVM часто работает поверх RAID

RAID дает отказоустойчивость, LVM добавляет гибкость управления и масштабирования.



Перед запуском приложения всегда проверяют mount point

Неправильная точка монтирования может привести к потере данных или просто.

Пример архитектуры хранения для /srv/apps

/dev/sdb1 и /dev/sdc1 образуют программный RAID1
/dev/md0

/dev/md0 используется как Physical Volume для LVM
Volume Group vg_data получает пространство от PV
/dev/md0

Logical Volume lv_apps выделяется из vg_data

На lv_apps создается ext4 и монтируется в /srv/apps

Такой порядок соединяет отказоустойчивость RAID и
гибкость LVM

Первые команды показывают карту хранения

Команда `lsblk` показывает диски, разделы, LVM-тома и точки монтирования

Пример использования: `lsblk -f`

Команда `blkid` показывает UUID и тип файловой системы

Пример использования: `sudo blkid /dev/sdb1`

Команда `fdisk -l` показывает таблицу разделов и размеры дисков

Перед разрушительными командами сначала выполняют `lsblk` и проверяют выбранный диск

LVM и mdadm могут работать с диском или разделом

В учебной VM допустимо использовать целый выделенный диск, например `/dev/sdb`

На реальном сервере часто сначала создают GPT-раздел, например `/dev/sdb1`

Для RAID обычно используют одинаковые разделы на разных дисках: `/dev/sdb1` и `/dev/sdc1`

Для LVM можно выполнить `pvcreate /dev/sdb` или `pvcreate /dev/sdb1`

Главное правило: заранее выбрать единый вариант и не перепутать учебный диск с рабочим

2. LVM добавляет гибкость поверх обычных устройств

LVM (Logical Volume Manager) - подсистема управления логическими томами Linux

LVM позволяет объединять устройства в общий пул пространства

Из этого пула администратор выделяет логические тома нужного размера

Том можно расширить без переноса данных на другой диск

LVM удобен для /srv, /var, /home и каталогов приложений

LVM не заменяет backup и не защищает от удаления

PV, VG и LV образуют модель LVM

PV (Physical Volume) - физический том LVM на диске, разделе или RAID-устройстве

VG (Volume Group) - группа физических томов, общий пул пространства

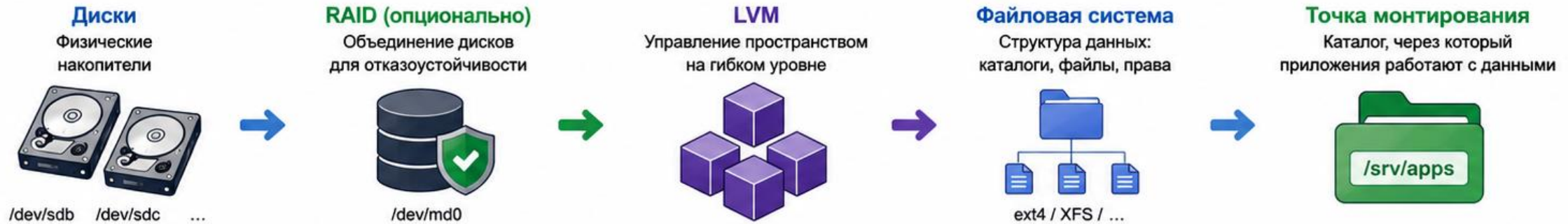
LV (Logical Volume) - логический том, который форматируется и монтируется

Файловая система создается на LV, а не на VG

Пример цепочки: /dev/md0 -> PV -> vg_data -> lv_apps -> ext4 -> /srv/apps

PHYSICAL VOLUME, VOLUME GROUP И LOGICAL VOLUME

Как LVM организует гибкое и отказоустойчивое хранение в Linux



LVM: ТРИ ОСНОВНЫХ ЭЛЕМЕНТА

PV – Physical Volume Физический том



PV

- Представляет собой диск или раздел
- Помечается для использования LVM
- Содержит метаданные LVM
- Может быть частью RAID-устройства

`/dev/sdb1` `/dev/md0` `/dev/nvme0n1p1`

VG – Volume Group Группа томов



vg_data

- Объединяет несколько PV в единое пул-пространство
- Пространство VG можно распределять между LV
- Легко расширяется добавлением новых PV
- Не имеет файловой системы

LV – Logical Volume Логический том



lv_apps

- Выделяется из свободного пространства VG
- Может быть любого размера
- Можно расширять или уменьшать без потери данных*
- На LV создаётся файловая система

ПОЛЕЗНЫЕ КОМАНДЫ



PV – Physical Volume

```
pvcreeate /dev/sdb1
pvdisplay
pvs
```



VG – Volume Group

```
vgcreate vg_data /dev/sdb1 /dev/md0
vgdisplay
vgs
```



LV – Logical Volume

```
lvcreate -L 50G -n lv_apps vg_data
lvdisplay
lvs
```



Файловая система

```
mkfs.ext4 /dev/vg_data/lv_apps
tune2fs -L APPS /dev/vg_data/lv_apps
```



Монтирование

```
mount /dev/vg_data/lv_apps /srv/apps
df -hT /srv/apps
```

ПРЕИМУЩЕСТВА LVM



- Гибкое управление пространством
- Легкое расширение томов «на лету»
- Снимки (snapshots) для резервного копирования
- Перенос томов между серверами
- Абстракция от физической структуры дисков

ПРИМЕР ЦЕПОЧКИ ХРАНЕНИЯ ДАННЫХ



ВАЖНО ПОМНИТЬ



- Удаление PV/VG/LV может привести к потере данных
- Всегда проверяйте устройства перед командами
- Держите резервные копии важных данных
- Используйте UUID при монтировании в `/etc/fstab`

* Расширение LV и файловой системы: сначала увеличиваем LV (`lvextend`), затем файловую систему (`resize2fs` для `ext4` или `xfs_growfs` для `XFS`).

Physical Volume создается перед включением устройства в LVM

Команда `pvcreate` записывает служебные метаданные LVM на устройство

Пример использования: `sudo pvcreate /dev/sdb`

Если используется раздел, пример будет таким: `sudo pvcreate /dev/sdb1`

Команда `pvs` показывает список Physical Volume и их принадлежность к VG

Пример проверки: `sudo pvs`

Команда `pvcreate` разрушительна для старых данных, поэтому перед ней обязательно проверить `lsblk -f`

Volume Group создается и расширяется только подготовленными PV

Команда `vgcreate` создает группу томов из одного или нескольких PV

Пример создания: `sudo vgcreate vg_data /dev/sdb`

Новый диск перед добавлением в VG сначала готовят командой `pvccreate`

Правильная последовательность: `sudo pvccreate /dev/sdc`, затем `sudo vgextend vg_data /dev/sdc`

Команды `pvs` и `vgs` проверяют, что PV добавлен и свободное место выросло

Ошибка `vgextend` часто означает, что устройство еще не является Physical Volume

Logical Volume получает имя, размер и назначение

Команда `lvcreate` выделяет логический том из свободного места Volume Group

Пример использования: `sudo lvcreate -n lv_apps -L 10G vg_data`

Команда `lvs` показывает созданные Logical Volume и их размер

Том доступен как `/dev/vg_data/lv_apps` или `/dev/mapper/vg_data-lv_apps`

На LV нужно создать файловую систему перед монтированием

Имя LV должно отражать назначение данных: `lv_apps`.

Форматирование и монтирование делают LV рабочим каталогом

Команда `mkfs.ext4` создает файловую систему `ext4` на логическом томе

Пример использования: `sudo mkfs.ext4 /dev/vg_data/lv_apps`

Команда `mkdir` создает точку монтирования

Пример использования: `sudo mkdir -p /srv/apps`

Команда `mount` подключает том к дереву каталогов

Пример использования: `sudo mount /dev/vg_data/lv_apps /srv/apps`

Расширение LVM выполняется на двух разных уровнях

Сначала расширяют сам Logical Volume

Пример использования: `sudo lvextend -L +5G`

`/dev/vg_data/lv_apps`

Затем расширяют файловую систему внутри тома

Для ext4 применяется команда: `sudo resize2fs`

`/dev/vg_data/lv_apps`

Команда `df -h /srv/apps` подтверждает новый размер файловой системы

Если расширить только LV, приложение может не увидеть новое место

Опция -r упрощает расширение, но не отменяет понимание уровней

Команда `lvextend -r` пытается расширить LV и файловую систему одной операцией

Пример использования: `sudo lvextend -r -L +5G /dev/vg_data/lv_apps`

Для обучения полезнее сначала выполнить `lvextend`, затем `resize2fs` отдельно

Так студент видит разницу между размером LV и размером файловой системы

Расширение и уменьшение - разные операции, их нельзя считать симметричными

Уменьшение томов требует отдельной осторожной процедуры

ext4 можно уменьшать только по отдельной процедуре с проверкой файловой системы

Обычно уменьшение ext4 требует размонтирования тома

XFS штатными средствами не уменьшается

Нельзя применять к уменьшению те же команды, что и к расширению

Перед уменьшением всегда нужен бэкап и проверенный план отката

В базовой лаборатории лучше отрабатывать расширение, а не уменьшение

XFS расширяется только в смонтированном состоянии

XFS - журналируемая файловая система, часто применяемая на серверах

Команда `xfs_growfs` получает точку монтирования, а не устройство

Пример использования: `sudo xfs_growfs /srv/apps`

Команда `df -T /srv/apps` показывает тип файловой системы

Команда `findmnt /srv/apps` подтверждает, что каталог действительно смонтирован

XFS поддерживает расширение, но не штатное уменьшение

3. Snapshot фиксирует состояние тома на момент времени

LVM snapshot - снимок Logical Volume для кратковременной фиксации состояния данных
Snapshot полезен перед обновлением, миграцией или резервным копированием

Он хранит измененные блоки, а не полную копию исходного тома

Snapshot находится на той же системе хранения и не спасает от потери сервера

Если пространство snapshot закончится, снимок станет непригодным

Snapshot не является полноценной резервной копией

Размер snapshot - это место для изменений после снимка

Команда `lvcreate -s` создает snapshot от исходного Logical Volume

Пример использования: `sudo lvcreate -s -n snap_apps -L 2G /dev/vg_data/lv_apps`

Размер 2G не означает полную копию тома размером 2G

Эти 2G используются для хранения блоков, измененных после создания snapshot

Команда `lvs -o lv_name,origin,lv_size,data_percent` показывает заполнение snapshot

Поле Data% помогает понять, не приближается ли

Snapshot обычно дает crash-consistent состояние

Crash-consistent копия похожа на состояние диска
после внезапного отключения питания

Файловая система может восстановиться по журналу,
но приложение могло не завершить запись

База данных может иметь незавершенную транзакцию
или данные в кэше

Application-consistent копия требует согласования с
приложением или СУБД

Для файловой системы может применяться fsfreeze,
для базы данных - штатный backup или quiesce

Snapshot без согласования нельзя автоматически

Правильный сценарий snapshot согласует данные перед копированием

Сначала приложение переводят в согласованное состояние или приостанавливают запись

Затем сбрасывают буферы или используют механизм backup приложения

После этого создают snapshot командой `lvcreate -s`

Работу приложения возобновляют сразу после создания snapshot

Данные копируют из snapshot, а не из активно изменяемого тома

После завершения копирования snapshot удаляют командой `sudo lvremove /dev/vg data/snap apps`

4. RAID повышает отказоустойчивость дисковой подсистемы

RAID (Redundant Array of Independent Disks) - массив из нескольких дисков

RAID1 зеркалирует данные на два диска и прост для небольших серверов

RAID5 использует распределенную четность, экономит емкость, но имеет штраф записи

RAID10 сочетает зеркалирование и чередование, быстрее и устойчивее при нагрузке

RAID не защищает от удаления файлов, ошибки администратора или шифровальщика

RAID и backup решают разные задачи и должны

RAID1, RAID5 и RAID10

Сравнение уровней RAID по отказоустойчивости, полезной ёмкости и типовым сценариям

КЛЮЧЕВЫЕ ОТЛИЧИЯ



RAID1 — самое простое зеркало



RAID5 — экономит место, но есть штраф записи



RAID10 — сочетает скорость и отказоустойчивость



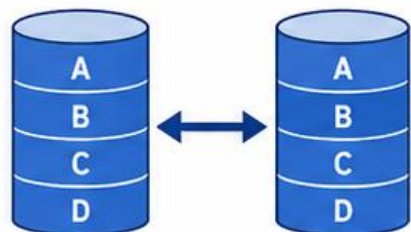
RAID10 может пережить несколько отказов, если они не в одной зеркальной паре



RAID5 рискован при долгом rebuild больших дисков

RAID1

ЗЕРКАЛО (MIRROR)



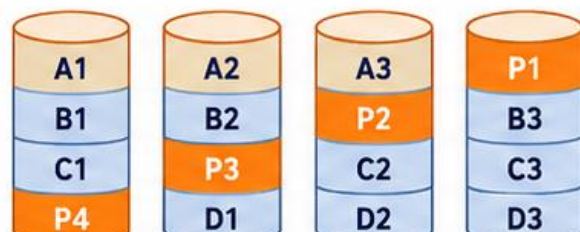
Диск 1

Диск 2

- Минимум: 2 диска
- Полезная ёмкость: 50%
- Отказ: обычно 1 диск
- Сильная сторона: простое зеркало

RAID5

ЧЕРЕДОВАНИЕ С ПАРИТЕТОМ



Диск 1

Диск 2

Диск 3

Диск 4

- Минимум: 3 диска
- Полезная ёмкость: $N - 1$ дисков
- Отказ: 1 диск
- Сильная сторона: баланс ёмкости и защиты

RAID10

ЗЕРКАЛЬНЫЕ ПАРЫ + ЧЕРЕДОВАНИЕ



Диск 1

Диск 2

Диск 3

Диск 4

- Минимум: 4 диска
- Полезная ёмкость: 50%
- Отказ: зависит от пар
- Сильная сторона: высокая производительность

СРАВНЕНИЕ

Уровень	Мин. дисков	Полезная ёмкость	Допустимый отказ	Типовое применение
RAID1	2	50%	1 диск	Небольшие серверы, ОС, критичные данные
RAID5	3	$(N-1)/N$	1 диск	Файловые хранилища, где важна ёмкость
RAID10	4	50%	1+ при разных парах	Базы данных, виртуализация, нагруженные сервисы

ЧТО ВАЖНО ПОМНИТЬ



RAID $\neq$ backup



Rebuild нагружает диски



Чем больше диски, тем дольше восстановление



Для интенсивной нагрузки чаще выбирают RAID1 или RAID10

ТИПОВАЯ ПРОВЕРКА В LINUX

```
cat /proc/mdstat
```

состояние массива

```
mdadm --detail /dev/md0
```

подробности RAID

```
lsblk
```

видно диски и md-устройства



RAID повышает отказоустойчивость, но не заменяет резервное копирование.

Выбор RAID зависит от емкости, скорости и допустимого отказа

RAID1: минимум 2 диска, полезная емкость равна размеру одного диска, обычно выдерживает отказ 1 диска

RAID5: минимум 3 диска, полезная емкость равна $(N - 1)$ дискам, выдерживает отказ 1 диска

RAID10: минимум 4 диска, полезная емкость равна $N / 2$ дисков

RAID10 может пережить несколько отказов только если они не попали в одну зеркальную пару

RAID5 рискован при длительном восстановлении больших дисков. потому что второй отказ приведет к

Состояния RAID в Linux нужно читать по реальному выводу

В /proc/mdstat часто встречаются состояния active, clean, degraded, recovering и resyncing

Строка [UU] означает, что оба участника зеркала работают

Строка [U_] означает, что один участник отсутствует или отказал

Слова recovery, resync и reshape показывают разные операции восстановления или перестроения

Обобщенный термин rebuilding полезен по смыслу, но в Linux вывод может называться иначе

Во время восстановления нагрузка на диски выше, а

mdadm создает массив, но синхронизацию нужно дождаться

mdadm - утилита управления программными RAID-массивами Linux

Пример RAID1: `sudo mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb /dev/sdc`

Команда `cat /proc/mdstat` показывает краткое состояние массива

Пример контроля: `watch cat /proc/mdstat`

Команда `sudo mdadm --detail /dev/md0` показывает подробности массива

До завершения `recover` или `resync` не стоит специально отключать диск для демонстрации отказа

После создания RAID проверяют автоматическую сборку

Созданный `/dev/md0` должен собираться после перезагрузки учебной VM

Команда `sudo mdadm --detail --scan` показывает строку конфигурации массива

Файл `/etc/mdadm/mdadm.conf` хранит параметры автосборки на Debian и Ubuntu

Команда `sudo update-initramfs -u` обновляет `initramfs` после изменения конфигурации `mdadm`

После перезагрузки проверяют `cat /proc/mdstat`, `sudo mdadm --detail /dev/md0` и `findmnt /srv/apps`

Точные действия могут отличаться в зависимости от

RAID можно использовать как Physical Volume для LVM

После создания /dev/md0 администратор может создать на нем файловую систему напрямую

Другой вариант - использовать /dev/md0 как Physical Volume для LVM

Пример использования: `sudo pvcreate /dev/md0`

Затем создается VG: `sudo vgcreate vg_data /dev/md0`

Так RAID обеспечивает отказоустойчивость дисков, а LVM - гибкое выделение `lv_apps`, `lv_logs` и других томов

Этот вариант ближе к реальной серверной архитектуре, чем одиночный диск без RAID

5. SMART помогает оценить здоровье физического накопителя

SMART (Self-Monitoring, Analysis and Reporting Technology) - система самодиагностики накопителей

Команда `smartctl` выводит атрибуты диска и результаты самопроверок

Пример использования для SATA/SAS: `sudo smartctl -a /dev/sda`

Для NVMe устройство может называться `/dev/nvme0`, пример: `sudo smartctl -a /dev/nvme0`

Reallocated sectors, pending sectors и ошибки чтения требуют внимания

Для `smartctl` обычно нужен пакет `smartmontools`

В VirtualBox SMART может быть недоступен

Гостевая ОС в VirtualBox видит виртуальный, а не физический накопитель

Виртуальный диск часто не передает реальные SMART-атрибуты

Команда `smartctl` может показать ограниченную информацию или сообщение о неподдерживаемом устройстве

В лаборатории `smartctl` изучается как инструмент реального сервера

На физическом сервере SMART нужно сопоставлять с моделью диска, журналами и данными контроллера

df, du и inode отвечают на разные вопросы

df показывает использование файловой системы целиком

Пример использования: `df -h /srv/apps`

du показывает объем файлов, доступных через выбранный каталог

Пример использования: `sudo du -xhd1 /var`

df -i показывает использование inode

inode - структура метаданных объекта файловой системы: владельца, прав, времени, размера и ссылок на блоки данных

inode заканчиваются не только у обычных файлов

inode расходуют файлы, каталоги и символические ссылки

Имя файла хранится в записи каталога, а не непосредственно в inode

Миллионы мелких файлов могут закончить inode раньше, чем закончится место в гигабайтах

Команда `df -i` показывает процент использования inode

Пример оценки количества файлов: `find /path -xdev -type f | wc -l`

Если inode закончились, удаление больших файлов

df и du расходятся, если удаленный файл открыт процессом

Файл может быть удален из каталога, но процесс продолжает держать его открытым
du уже не видит такой файл, потому что имени в каталоге нет

df продолжает показывать занятое место, пока процесс не закроет файл

Команда `sudo lsof +L1` показывает открытые удаленные файлы

Дополнительная проверка: `sudo lsof | grep deleted`

Место освобождается после закрытия файла

процессом или перезапуска соответствующей службы

find и du на больших деревьях создают нагрузку

Команды find и du читают много метаданных файловой системы

На каталоге с миллионами файлов они могут работать долго

На рабочем сервере такие команды запускают осознанно и желательно вне пикового времени

Ключ -xdev помогает не уходить в другие смонтированные файловые системы

Пример безопасного уровня просмотра: `sudo du -xhd1 /var`

Администратор сначала ищет крупный верхний

6. UUID делает монтирование устойчивым к смене имен дисков

UUID (Universally Unique Identifier) - уникальный идентификатор файловой системы

Имена `/dev/sdb` и `/dev/sdc` могут измениться после перезагрузки или изменения порядка дисков

Запись в `/etc/fstab` по UUID надежнее, чем запись по имени диска

Команда `blkid` показывает UUID файловой системы

Пример использования: `sudo blkid`

`/dev/vg_data/lv_apps`

Команда `lsblk -f` помогает сопоставить устройство,

UUID, тип ФС и точку монтирования

nofail подходит не для каждого тома

Параметр `nofail` разрешает системе продолжить загрузку, если том не смонтировался

Для необязательного архива это может быть полезно:

```
UUID=... /mnt/archive ext4 defaults,nofail 0 2
```

Для критического каталога приложения `nofail` может скрыть проблему

Пример для `/srv/apps` лучше делать без `nofail`: `UUID=...`
`/srv/apps ext4 defaults 0 2`

Если критический том не смонтирован, отказ должен быть замечен до запуска приложения

Иначе приложение может начать писать данные на

Монтирование скрывает файлы, которые уже лежали в каталоге

Если в `/srv/apps` уже были файлы, после `mount` другой файловой системы они станут невидимыми

Это не удаление: файлы снова появятся после `umount`

Типовая ошибка: приложение записало данные в `/srv/apps` на корневом разделе, затем администратор смонтировал новый пустой LV

Перед первым монтированием нужно проверить каталог командой `sudo ls -la /srv/apps`

Если данные уже есть, их нужно корректно перенести на новый том

Команда `findmnt /srv/apps` показывает, что сейчас

Проверка `/etc/fstab` выполняется до перезагрузки

После редактирования `/etc/fstab` нельзя сразу перезагружать сервер

Команда `sudo systemctl daemon-reload` перечитывает unit-зависимости `systemd` после изменения `fstab`

Команда `sudo mount -a` пытается смонтировать все записи из `/etc/fstab`

Команда `findmnt --verify` проверяет корректность описания монтирования

Команда `findmnt /srv/apps` подтверждает фактический mount point

Команда `df -hT /srv/apps` показывает тип ФС и

7. Заполнение диска проявляется через сбои служб

Службы могут перестать писать журналы, временные файлы или данные приложения

Команда `df -h` показывает заполненные файловые системы

Команда `du` помогает найти источник роста данных внутри каталога

Журналы `systemd` можно оценить командой `journalctl -disk-usage`

Освобождать место нужно осознанно: ротировать журналы, удалять временное или расширять том

Если `df` и `du` расходятся, нужно проверить открытые

Отказ диска требует точной идентификации устройства

Нельзя заменять диск только по имени `/dev/sdb`,
потому что имена могут измениться

Диск идентифицируют по серийному номеру, WWN,
модели и физическому слоту

Команда `lsblk -o`

`NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS,MODEL,SERIAL`
показывает важные признаки

Для реального сервера дополнительно используют
данные контроллера, BMC или документацию стойки

Сначала проверяют `/proc/mdstat` и `mdadm --detail`,
затем сопоставляют отказавший участник с

ДИАГНОСТИКА ПРОБЛЕМ ДИСКОВОЙ ПОДСИСТЕМЫ

Как быстро найти и решить проблемы с дисками, RAID, LVM, файловыми системами и монтированием

1. СИМПТОМЫ И ВОЗМОЖНЫЕ ПРИЧИНЫ

Диск заполнен



- Большие файлы и логи
- Копирование данных
- Нет ротации журналов
- Неочищенный кэш

Не хватает inode



- Много мелких файлов
- Распаковка архивов
- Временные файлы
- Некорректная очистка

Медленная работа



- Высокая нагрузка I/O
- Фрагментация
- Медленные диски
- Проблемы RAID/LVM

RAID в состоянии degraded



- Отказ диска
- Кабель/контроллер
- Сбой синхронизации
- Нехватка места для rebuild

Ошибка монтирования



- Неверный UUID/путь
- Повреждена ФС
- Неправильные права
- Нет зависимостей (LVM/RAID)

LVM том не активен



- VG не найден
- PV отсутствует
- Повреждены метаданные
- Нет свободного места

2. КОМАНДЫ ДЛЯ ДИАГНОСТИКИ

Что проверяем	Команда	Пример	Что покажет
Диски и разделы	<code>lsblk -f</code>	<code>lsblk -f</code>	Дерево устройств, тип ФС, UUID, точки монтирования
Использование места	<code>df -hT</code>	<code>df -hT</code>	Свободное место по файловым системам и их тип
Использование inode	<code>df -i</code>	<code>df -i</code>	Свободные inode, процент использования
Размер каталогов	<code>du -sh /*</code>	<code>du -sh /*</code>	Какие каталоги занимают больше всего
RAID статус	<code>cat /proc/mdstat</code>	<code>cat /proc/mdstat</code>	Состояние массивов: active, degraded, rebuilding
Детали RAID	<code>sudo mdadm --detail /dev/md0</code>	<code>sudo mdadm --detail /dev/md0</code>	Уровень, диски, состояние, время на rebuild
LVM информация	<code>pvs; vgs; lvs</code>	<code>pvs; vgs; lvs</code>	PV, VG, LV: размеры, свободное место, статусы
Журнал событий	<code>journalctl -xe</code>	<code>journalctl -xe</code>	Ошибки системы, дисков, RAID, LVM, монтирования
SMART дисков	<code>sudo smartctl -a /dev/sda</code>	<code>sudo smartctl -a /dev/sda</code>	Состояние диска, ошибки, температура, прогноз
Проверка ФС	<code>sudo fsck -n /dev/sda1</code>	<code>sudo fsck -n /dev/sda1</code>	Проверка файловой системы (без исправлений)

3. ТИПИЧНЫЕ ПРОБЛЕМЫ И ЧТО ДЕЛАТЬ

Проблема	Как определить	Что делать
Диск заполнен	<code>df -hT</code> → Use% = 100% <code>du -sh /*</code> → найти большой каталог	• Очистить /var/log, кэш, временные файлы • Удалить ненужные данные • Настроить ротацию логов (logrotate)
Не хватает inode	<code>df -i</code> → IUse% = 100% <code>du -ahx --inodes sort -nr head</code>	• Найти каталоги с большим числом файлов • Удалить лишние мелкие файлы • Использовать ФС с большим inode ratio
RAID degraded	<code>cat /proc/mdstat</code> <code>mdadm --detail /dev/md0</code>	• Определить отказавший диск • Заменить диск • Запустить восстановление (rebuild)
LVM том недоступен	<code>pvs / vgs / lvs</code> → том не отображается <code>dmesg grep -i lvm</code>	• Проверить наличие PV и VG • Активировать VG: <code>vgchange -ay</code> • Восстановить метаданные LVM
Ошибка монтирования	<code>mount -a</code> → ошибка <code>dmesg tail</code>	• Проверить UUID и путь в /etc/fstab • Проверить ФС: <code>fsck</code> • Исправить права и зависимости
Медленная работа	<code>iostat -xz 1</code> <code>iotop -oP</code> <code>vmstat 1</code>	• Проверить нагрузку и I/O • Найти «тяжёлые» процессы • Рассмотреть апгрейд дисков/RAID/LVM

4. ОБЩИЙ АЛГОРИТМ ДИАГНОСТИКИ

- 1 Проверить симптомы и получить жалобы пользователя
- 2 Проверить место и inode: `df -hT`, `df -i`
- 3 Проверить состояние RAID и LVM: `/proc/mdstat`, `pvs`, `vgs`, `lvs`
- 4 Проверить точки монтирования: `lsblk -f`, `mount`
- 5 Проверить журналы: `journalctl -xe`, `dmesg`
- 6 Выполнить действия по устранению
- 7 Повторно проверить систему

ВАЖНО ПОМНИТЬ



- Всегда делайте резервные копии важных данных
- Проверяйте SMART и состояние RAID регулярно
- Не запускайте fsck на смонтированной файловой системе
- При отказе диска – действуйте быстро
- Следите за свободным местом (оставляйте 10–20%)
- Планируйте контроль: cron + отчёты

ПОЛЕЗНЫЕ КОМАНДЫ НА ЗАМЕТКУ

<code>watch -n 1 df -hT</code>	мониторинг места	<code>sudo vgchange -ay</code>	активировать VG
<code>sudo dmesg -T less</code>	сообщения ядра	<code>sudo lsdf +L1</code>	найти удалённые, но занятые файлы
<code>sudo mdadm --monitor --scan</code>	мониторинг RAID	<code>history grep -i mount</code>	история монтирований

КОГДА НУЖНА ПРОВЕРКА ФС

<code>fsck -n /dev/sdX1</code>	только проверка
<code>fsck -y /dev/sdX1</code>	автоматическое исправление



Выполнять только на НЕсмонтированной ФС!

Единый сценарий практики связывает все уровни хранения

Добавить в учебную VM два диска и проверить их через lsblk

Создать RAID1, дождаться синхронизации и проверить /proc/mdstat

Использовать /dev/md0 как PV, создать vg_data и lv_apps

Создать ext4, смонтировать /srv/apps и настроить /etc/fstab по UUID

Создать snapshot, проверить Data% и удалить снимок после использования

Смоделировать отказ участника RAID, восстановить

Итоги лекции

Linux-хранилище нужно понимать как цепочку уровней, а не как один диск

RAID дает отказоустойчивость дисков, LVM дает гибкое распределение пространства

Snapshot полезен кратковременно, но требует понимания консистентности данных

df, du, df -i, lsof, SMART, mdadm и /proc/mdstat проверяют разные уровни проблемы

UUID и проверка /etc/fstab защищают сервер от ошибок после перезагрузки

Перед заменой диска нужно идентифицировать

Контрольные вопросы

Чем диск, раздел, файловая система и точка монтирования отличаются друг от друга?

Почему RAID и LVM часто используют вместе?

Зачем перед `vgextend` выполнять `pvcreate`?

Почему snapshot нельзя считать полноценным backup?

Чем crash-consistent копия отличается от application-consistent?

Как по `/proc/mdstat` понять отказ участника RAID1?

Почему `nofail` опасен для критического каталога приложения?

Почему `df` может показывать занятое место, которое